

Investigación TESTING:

- Planes de Prueba:

➤ ¿Qué es un plan de pruebas de Software? ¿Cuál es el ciclo de vida de pruebas de Software? Realice un gráfico del ciclo de vida.

**El plan de pruebas es un documento detallado sobre qué se va a probar, cómo se va a probar, cuándo se va a probar y quién será responsable de hacerlo. Funciona como una hoja de ruta para guiar al equipo en la garantía de calidad QA.**

**Su ciclo de vida es el siguiente:**

#### **1.Análisis de Requisitos:**

**El equipo de QA revisa y analiza los requisitos funcionales y no funcionales del sistema para identificar qué necesita ser probado y detectar ambigüedades o vacíos técnicos de forma temprana.**

#### **2.Planificación de Pruebas:**

**Se redacta el plan de pruebas. Se estiman esfuerzos, costos, plazos, herramientas necesarias y asignación de roles dentro del equipo de pruebas.**

#### **3.Diseño y Desarrollo de Casos de Prueba:**

**Se redactan los casos de prueba detallados, se preparan los scripts de automatización y se generan los datos de prueba necesarios.**

#### **4.Configuración del Entorno de Pruebas:**

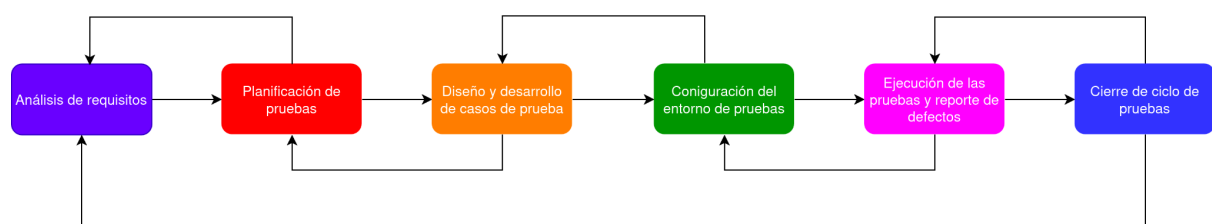
**Se instala y configura el entorno (servidores, bases de datos, versiones de software) donde se ejecutarán las pruebas, asegurando que replique de la forma más fiel posible el entorno de producción.**

#### **5.Ejecución de Pruebas y Reporte de Defectos:**

**Se ejecutan los casos de prueba. Las discrepancias entre el resultado esperado y el real se reportan como defectos o bugs en la herramienta de seguimiento, y se re-prueban una vez corregidos por los desarrolladores.**

#### **6.Cierre del Ciclo de Pruebas:**

**Se evalúan las métricas de calidad y se elabora el informe final de pruebas para validar si el software está listo para despliegue.**



- Tipos de Prueba:

➤ Realizar un cuadro con los tipos de prueba, donde se describa cada uno y especifique el objetivo de su ejecución.

| Tipo de Prueba                     | Descripción                                                                                                                 | Objetivo de Ejecución                                                                                            |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Pruebas Unitarias</b>           | Evalúan componentes mínimos de código (funciones, métodos o clases) de forma aislada.                                       | Validar que cada unidad lógica funcione correctamente por separado y detectar errores en etapas tempranas.       |
| <b>Pruebas de Integración</b>      | Verifican la comunicación e intercambio de datos entre módulos combinados o servicios externos (APIs, bases de datos).      | Identificar fallos en las interfaces y en el flujo de datos entre componentes dependientes.                      |
| <b>Pruebas de Sistema (E2E)</b>    | Prueban la aplicación completa e integrada simulando el flujo de negocio real de punta a punta.                             | Comprobar que el sistema completo cumple con los requisitos funcionales y técnicos especificados.                |
| <b>Pruebas de Aceptación (UAT)</b> | Ejecutadas generalmente por usuarios finales o stakeholders en un entorno similar al productivo.                            | Confirmar que el software satisface las necesidades del negocio y validar si está listo para salir a producción. |
| <b>Pruebas de Regresión</b>        | Reejecución de casos de prueba previos tras introducir cambios en el código, correcciones de bugs o nuevas funcionalidades. | Garantizar que las modificaciones recientes no rompieron funcionalidades existentes que ya funcionaban bien.     |

|                                                 |                                                                                                                             |                                                                                                                    |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Pruebas de Humo<br/>(Smoke Testing)</b>      | Conjunto mínimo y rápido de pruebas que revisa las funciones más críticas de una nueva versión.                             | Confirmar la estabilidad básica de la compilación para decidir si el entorno es apto para pruebas más exhaustivas. |
| <b>Pruebas de Rendimiento<br/>(Performance)</b> | Miden el comportamiento, velocidad de respuesta, estabilidad y uso de recursos bajo diferentes volúmenes de carga o estrés. | Asegurar que el sistema soporte la demanda de tráfico esperada sin degradarse o colapsar.                          |
| <b>Pruebas de Seguridad</b>                     | Analizan vulnerabilidades, controles de acceso, cifrado e inyección de datos para detectar brechas.                         | Proteger el sistema y los datos contra accesos no autorizados, ataques maliciosos o fugas de información.          |
| <b>Pruebas de Usabilidad</b>                    | Evalúan la interfaz de usuario, la navegación y la facilidad de aprendizaje del producto con usuarios representativos.      | Medir la ergonomía, accesibilidad e intuición del software para optimizar la experiencia de usuario (UX).          |

- Patrones de Automatización de Pruebas.

➤ ¿Qué son los patrones de automatización de pruebas de software (Modelo de Objeto de Páginas “POM”, Pruebas Basadas en Datos, Desarrollo Impulsado por Comportamiento “BDD”, Pruebas de Carga y Pruebas de Stress)?. Realizar una explicación de cada uno de ellos.

**Modelo de Objeto de Páginas:** Es un patrón de diseño para pruebas de interfaz de usuario donde cada pantalla se representa como una clase de código independiente que encapsula sus elementos y acciones, lo que evita duplicar código y simplifica el mantenimiento cuando la interfaz gráfica cambia.

**Pruebas Basadas en Datos:** Es una técnica que desacopla la lógica del script de los datos de entrada, cargándolos desde fuentes externas (como CSV, JSON o Excel) para ejecutar el mismo flujo una y otra vez con múltiples combinaciones sin reescribir código.

**Desarrollo Impulsado por Comportamiento:** Es un enfoque colaborativo que define el comportamiento del sistema en lenguaje natural estructurado (*Dado que / Cuando / Entonces*), permitiendo que perfiles técnicos y de negocio compartan una misma especificación que luego se ejecuta directamente como prueba automatizada.

**Pruebas de Carga:** Consisten en simular un volumen previsto de usuarios o transacciones concurrentes para medir los tiempos de respuesta, latencias y consumo de recursos bajo condiciones normales y picos esperados de operación.

**Pruebas de Estrés:** Consisten en exigir al sistema deliberadamente más allá de sus límites operativos para identificar su punto de quiebre, analizando cómo falla, si corrompe información y con qué rapidez logra recuperarse de un colapso.